Summarize

Chapter 9. Functions and Event-Driven Processing

So far, we have examined design for systems with long-running computation. The servers that handle user requests are always up and running. This pattern is the right one for many applications that are under heavy load, keep a large amount of data in memory, or require some sort of background processing. However, there is a class of applications that might only need to temporarily come into existence to handle a single request, or simply need to respond to a specific event. This style of request or event-driven application design has flourished recently as large-scale public cloud providers have developed *function-as-a-service* (FaaS) products. More recently, FaaS implementations have also emerged running on top of cluster orchestrators in private cloud or physical environments. This chapter describes emerging architectures for this new style of computing. In many cases, FaaS is a component in a broader architecture rather than a complete solution.

NOTE

Oftentimes, FaaS is referred to as *serverless* computing. And while this is true (you don't see the servers in FaaS), it's worth differentiating between event-driven FaaS and the broader notion of serverless computing. Indeed, serverless computing can apply to a wide variety of computing services; for example, a multitenant container orchestrator (container-as-a-service) is serverless but not event-driven. Conversely, an open source FaaS running on a cluster of physical machines that you own and administer is event-driven but not serverless. Understanding this distinction enables you to determine when event-driven, serverless, or both is the right choice for your application.

Determining When FaaS Makes Sense

As with many tools for developing a distributed system, it can be tempting to see a particular solution like event-driven processing as a universal hammer. However, the truth

is that it is best suited to a particular set of problems. Within a particular context it is a powerful tool, but stretching it to fit all applications or systems will lead to overly complicated, brittle designs. Especially since FaaS is a relatively new computing tool, before discussing specific design patterns, it is worth discussing the benefits, limitations, and optimal situations for employing event-driven computing.

The Benefits of FaaS

The benefits of FaaS are primarily for the developer. It dramatically simplifies the distance from code to running service. Because there is no artifact to create or push beyond the source code itself, FaaS makes it simple to go from code on a laptop or web browser to running code in the cloud.

Likewise, the code that is deployed is managed and scaled automatically. As more traffic is loaded onto the service, more instances of the function are created to handle that increase in traffic. If a function fails due to application or machine failures, it is automatically restarted on some other machine.

Finally, much like containers, functions are an even more granular building block for designing distributed systems. Functions are stateless, and thus any system you build on top of functions is inherently more modular and decoupled than a similar system built into a single binary. But, of course, this is also the challenge of developing systems in FaaS. The decoupling is both a strength and a weakness. The following section describes some of the challenges that come from developing systems using FaaS.

The Challenges of FaaS

As described in “The Benefits of FaaS”, developing systems using FaaS forces you to strongly decouple each piece of your service. Each function is entirely independent. The only communication is across the network, and each function instance cannot have local memory, requiring all states to be stored in a storage service. This forced decoupling can improve the agility and speed with which you can develop services, but it can also significantly complicate the operations of the same service.

In particular, it is often quite difficult to obtain a comprehensive view of your service, determine how the various functions integrate with one another, and understand when things go wrong, and why they go wrong. Additionally, the request-based and serverless nature of functions means that certain problems are quite difficult to detect. As an example, consider the following functions:

- `functionA()` which calls `functionB()`
- `functionB()` which calls `functionC()`
- `functionC()` which calls back to `functionA()`

Now consider what happens when a request comes into any of these functions: it kicks off an infinite loop that only terminates when the original request times out (and possibly not even then) or when you run out of money to pay for requests in the system. Obviously, the above example is quite contrived, but it is actually quite difficult to detect in your code. Since each function is radically decoupled from the other functions, there is no real representation of the dependencies or interactions between different functions. These problems are not unsolvable, and I expect that as FaaSs mature, more analysis and debugging tools will provide a richer experience to understand how and why an application comprised of FaaS is performing the way that it does.

For now, when adopting FaaS, you must be vigilant to adopt rigorous monitoring and alerting for how your system is behaving so that you can detect situations and correct them before they become significant problems. Of course, the complexity introduced by monitoring flies somewhat in the face of the simplicity of deploying to FaaS, which is friction that your developers must overcome.

The Need for Background Processing

FaaS is inherently an event-based application model. Functions are executed in response to discrete events that occur and trigger the execution of the functions. Additionally, because of the serverless nature of the implementation of these services, the runtime of any particular function instance is generally time bounded. This means that FaaS is usually a poor fit for situations that require processing. Examples of such background processing might be transcoding a video, compressing log files, or other sorts of low-priority, long-running computations. In many cases, it is possible to set up a scheduled trigger that synthetically generates events in your functions on a particular schedule. Though this is a good fit for responding to temporal events (e.g., firing a text-message alarm to wake someone up), it is still not sufficient infrastructure for generic background processing. To achieve that, you need to launch your code in an environment that supports long-running processes. And this generally means switching to a pay-per-consumption rather than pay-per-request model for the parts of your application that do background processing.

The Need to Hold Data in Memory

In addition to the operational challenges, there are some architectural limitations that make FaaS ill-suited for some types of applications. The first of these limitations is the need to have a significant amount of data loaded into memory in order to process user requests. There are a variety of services (e.g., serving a search index of documents) that require a great deal of data to be loaded in memory in order to service user requests. Even with a relatively fast storage layer, loading such data can take significantly longer than the desired time to service a user request. Because with FaaS, the function itself may be dynamically spun up in response to a user request *while the user is waiting*, the need to load a lot of detail may significantly impact the latency that the user perceives while interacting with your service. Of course, once your FaaS has been created, it may handle a large number of requests, so this loading cost can be amortized across a large number of requests. But if you have a sufficient number of requests to keep a function active, then it's likely you are overpaying for the requests you are processing.

The Costs of Sustained Request-Based Processing

The cost model of public cloud FaaS is based on per-request pricing. This approach is great if you only have a few requests per minute or hour. In such a situation, you are idle most of the time, and given a pay-per-request model, you are only paying for the time when your service is actively serving requests. In contrast, if you service requests via a long-running service, either in a container or a virtual machine, then you are always paying for processor cycles that are largely sitting around waiting for a user request.

However, as a service grows, the number of requests that you are servicing grows to the point where you can keep a processor continuously active servicing user requests. At this point, the economics of a pay-per-request model start to become bad, and only get worse because the cost of cloud virtual machines generally decreases as you add more cores (and also via committed resources like reservations or sustained use discounts), whereas the cost per-request largely grows linearly with the number of requests.

Consequently, as your service grows and evolves, it's highly likely that your use of FaaS will evolve as well. One ideal way to scale FaaS is to run an open source FaaS that runs on a container orchestrator like Kubernetes. That way, you can still take advantage of the developer benefits of FaaS, while taking advantage of the pricing models of virtual machines.

Patterns for FaaS

In addition to understanding the trade-offs in deploying event-driven or FaaS architectures as part of your distributed system, understanding the best ways to deploy FaaS is critical to the design of a successful system. This section describes some canonical patterns for incorporating FaaS.

The Decorator Pattern: Request or Response Transformation

FaaS is ideal for deploying simple functions that can take an input, transform it into an output, and then pass it on to a different service. This general pattern can be used to augment or decorate HTTP requests to or from a different service. A basic illustration of this pattern is shown in Figure 9-1.

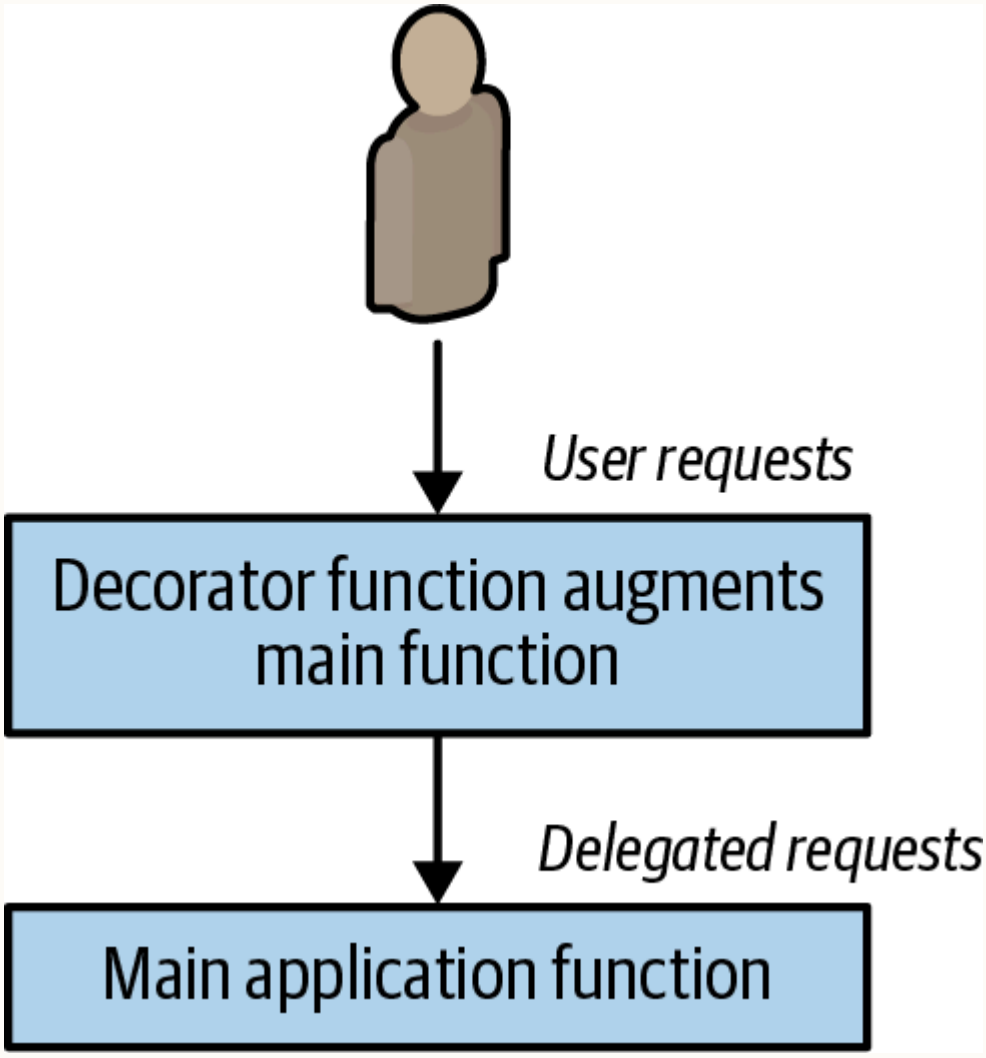


Figure 9-1. The decorator pattern applied to HTTP APIs

Interestingly, there are several analogies to this pattern in programming languages. In particular, the *decorator* pattern from Python is a close analogue for the services that a request or response decorator can perform. Because decoration transformations are generally stateless, and also because they are often added after the fact to existing code as

the service evolves, they are ideal services to implement via FaaS. Additionally, the lightness of FaaS means that you can experiment with a variety of different decorators before finally adopting one and pulling it more completely into the implementation of the service.

A great example of the value of the decorator pattern is adding defaults to the input to an HTTP RESTful API. In many cases in the API, there are fields whose values should have sane defaults if they are left empty. For example, you may want a field to default to `true`, but it's difficult to accomplish this in classical JSON, because the default value for a field is `null`, which is generally understood to be `false`. To resolve this, we can add defaulting logic either in the front of the API server or within the application code itself (e.g., `if (field == null) field = true`). However, both of these solutions are somewhat unappealing since the defaulting mechanism is fairly conceptually independent from the handling of the request. Instead, we can use the FaaS decorator pattern to transform the request in between the user and the service implementation.

Given the previous discussion of the adapter pattern in the single-node section, you may be wondering why we don't simply package this defaulting as an adapter container. And this is a totally reasonable approach, but it does mean that we are going to couple the scale of the defaulting service with the API service itself. The defaulting is actually a lightweight operation, and we are likely to need far fewer instances of it than the service itself to handle the load.

NOTE

For the examples in this chapter, we are going to use the `kubeless` FaaS framework. Kubeless is deployed on top of the Kubernetes container orchestration service. Assuming that you have provisioned a Kubernetes cluster, you can install Kubeless from its releases page. Once you have the `kubeless` binary installed, you can install it into your cluster with the following commands: `kubeless install`.

Kubeless installs itself as a native Kubernetes third-party API. This means that once it is installed, you can use the native `kubectl` command-line tool. For example, you can see deployed functions using `kubectl get functions`. Currently, you should have no functions deployed.

Hands On: Adding Request Defaulting Prior to Request Processing

To demonstrate the utility of the decorator pattern for FaaS, consider the task of adding default values to a RESTful function call if the values are missing. This is quite straight-

forward to do using FaaS. We'll write the defaulting function using the Python programming language:

```
# Simple handler function for adding default values
def handler(context):
  # Get the input value
  obj = context.json
  # If the 'name' field is not present, set it randomly
  if obj.get("name", None) is None:
    obj["name"] = random_name()
  # If the 'color' field is not present, set it to 'blue'
  if obj.get("color", None) is None:
    obj["color"] = "blue"
  # Call the actual API, potentially with the new default
  # values, and return the result
  return call_my_api(obj)
```

Save this function in a file named *defaults.py*. You obviously will want to update the `call_my_api` code so that it points to the actual API you want to call. Once you have finished writing the code, this defaulting function can be installed as a `kubeless` function using:

```
kubeless function deploy add-defaults \
    --runtime python27 \
    --handler defaults.handler \
    --from-file defaults.py \
    --trigger-http
```

If you want to test the handling of this function, you can also use the `kubeless` tool:

```
kubeless function call add-defaults --data '{"name": "foo"}'
```

The decorator pattern shows just how easy it is to adapt and extend existing APIs with additional features like validation or defaulting.

Handling Events

While most systems are request-driven, handling a steady stream of user and API requests, many other systems are more event-driven in nature. The differentiation, in my mind at least, between a request and an event has to do with the notion of *session*.

Requests are part of a larger series of interactions or sessions; generally each user request is part of a larger interaction with a complete web application or API. *Events*, as I see them, instead tend to be single-instance and asynchronous in nature. Events are important and need to be properly handled, but they are fired off from a main interaction and responded to some time later. Examples of events include a user signing up for a new service (which might trigger a welcome email), someone uploading a file to a shared folder (which might send notifications to everyone who has access to the folder), or even a machine being about to reboot (which might notify an operator or automated system to take appropriate action).

Because these events tend to be largely independent and stateless in nature, and because the rate of events can be highly variable, they are ideal candidates for event-driven and FaaS architectures. In this role, they are often deployed alongside a production application server as augmentation to the main user experience, or to handle some sort of reactive, background processing. Additionally, because new events are often dynamically added to the service, the lightweight nature of deploying functions is a good match for defining new event handlers. Likewise, because each event is conceptually independent, the forced decoupling of a functions-based system actually helps *reduce* the conceptual complexity by enabling a developer to focus on the steps required to handle just a single type of event.

A concrete example of integrating an event-based component to an existing service is implementing two-factor authentication. In this case, the event is the user logging into a service. The service can generate an event for this action, fire it into a function-based handler that takes the code and the user's contact information, and sends the two-factor code via text message.

Hands On: Implementing Two-Factor Authentication

Two-factor authentication requires that the user both have something that they know (e.g., a password) as well as something that they possess (e.g., a phone) to be able to log in to the system. Two-factor authentication is significantly more secure than passwords alone since it requires two different security compromises (a thief learning your password *and* a thief stealing your phone) to enable a true security problem.

When considering how to implement two-factor authentication, one of the challenges is how to handle the request to generate a random code and register it with the login service as well as send the text message. It is possible to add this code to the main login web server. But it is complicated and monolithic, and forces the act of sending a text

message, which can have some latency, to be inline with the code that renders the login web page. This latency produces a substandard user experience.

A better option is to register a FaaS to asynchronously generate the random number, register it with the login service, and send the number to the user's phone. In this way, the login server can simply fire an asynchronous web-hook request to a FaaS, and that FaaS can handle the somewhat slow and asynchronous task of registering the two-factor code and sending the text message.

To see how this works in practice, consider the following code:

```
def two_factor(context):
  # Generate a random six-digit code
  code = random.randint(100000, 999999)

  # Register the code with the login service
  user = context.json["user"]
  register_code_with_login_service(user, code)

  # Use the twillio library to send texts
  account = "my-account-sid"
  token = "my-token"
  client = twilio.rest.Client(account, token)

  user_number = context.json["phoneNumber"]
  msg = "Hello {} your authentication code is: {}.".format(user, code)
  message = client.api.account.messages.create(to=user_number,
                                  from_="+12065251212",
                                  body=msg)
  return {"status": "ok"}
```

We can then register this FaaS with `kubeless`:

```
kubeless function deploy add-two-factor \
    --runtime python27 \
    --handler two_factor.two_factor \
    --from-file two_factor.py \
    --trigger-http
```

This function can then be made asynchronously from client-side JavaScript whenever the user successfully provides their password. The web UX can then immediately display a

page to enter the code, and the user (once they receive the code as a text message) can supply it to the service, where the code has already been registered via our FaaS.

Again, developing a simple, asynchronous, event-based service that is triggered whenever a user logs in is made dramatically simpler using FaaS.

Event-Based Pipelines

There are some applications that are inherently easier to think about in terms of a pipeline of decoupled events. These event pipelines often resemble the flowcharts of old. They can be represented as a directed graph of connected event syncs. In the event pipeline pattern, each node is a different function or webhook, and the edges linking the graph together are HTTP or other network calls to the function/webhook. In general, there is no shared state between the different pieces of the pipeline, but there may be a context or other reference point that can be used to look up information in shared storage.

So, what is the difference between this type of pipeline and a "microservices" architecture? There are two central differences. The first is the main difference between functions in general and long-running services, which is that an event-based pipeline is by its very nature event-driven. Conversely, a microservices architecture features a collection of long-running services. Additionally, event-driven pipelines may be highly asynchronous and diverse in the things that they connect together. For example, while it is difficult to see how a human approving a ticket in a ticketing system like Jira could be integrated into a microservices application, it's quite easy to see how that event could be incorporated into an event-driven pipeline.

As an example of this, imagine a pipeline in which the original event is code being submitted into a source control system. This event then triggers a build. The build may take multiple minutes to complete, and when it does, it fires an event to a build analysis function. This function takes different actions if the build is successful or fails. If the build succeeds, a ticket is created for a human to approve it to be pushed to production. Once the ticket is closed, the act of closing is an event that triggers the actual push to production. If the build fails, a bug is filed on the failure, and the event pipeline terminates.

Hands On: Implementing a Pipeline for New User Signup

Consider the task of implementing a new user signup flow. When a new user account is created, there are certain things that are always done, such as sending a welcome email.

And there are some things that are optionally done, such as registering a user to receive product updates (sometimes known as "spam") via their email.

One approach to implementing this logic would be to place everything into a single monolithic *user-creation* server. However, this factoring means that a single team owns the entirety of the user-creation service, and that the entire experience is deployed as a single service. Both of these mean that it is more difficult to perform experiments or make changes to the user experience.

Consider, instead, implementing the user login experience as an event pipeline with a series of FaaS. In this factoring, the user-creation function is actually unaware of the details of what happens on user login. Instead, the main user-creation service simply has two lists:

- A list of required actions (e.g., sending a welcome mail)
- A list of optional actions (e.g., subscribing the user to a mailing list)

Each of these actions is also implemented as a FaaS, and the list of actions is actually just a list of webhooks. Consequently, the main user creation function looks like this:

```
def create_user(context):
  # For required event handlers, call them universally
  for key, value in required.items():
    call_function(value.webhook, context.json)

  # For optional event handlers, check and call them
  # conditionally
  for key, value in optional.items():
    if context.json.get(key, None) is not None:
      call_function(value.webhook, context.json)
```

Now we can also use FaaS to implement each of these handlers:

```
def email_user(context):
  # Get the user name
  user = context.json['username']

  msg = 'Hello {} thanks for joining my awesome service!".format(user)

  send_email(msg, contex.json['email])
```

```
def subscribe_user(context):
  # Get the user name
  email = context.json['email']
  subscribe_user(email)
```

Factored in this way, each FaaS is simple, containing only a few lines of code and focused on implementing one specific piece of functionality. This microservices-based approach is simple to write but might lead to complexity if we actually had to deploy and manage three different microservices. This is where FaaS can shine, since it makes it trivially easy to host these small code snippets. Additionally, by visualizing our user-creation flow as an event-driven pipeline, it is also straightforward to have a high-level understanding of what exactly happens on user login, simply by following the flow of the context through the various functions in the pipeline.

Summary

Functions as a service, or FaaS, is a powerful tool for easily hosting simple, scalable applications. As with any tool, the key to using it for maximal value lies in understanding what it is best at. The simplicity of functions as a service comes with restrictions that may limit ways in which your application can develop or libraries that your application can use. However, FaaS can also radically reduce your time to market. As you gain experience with distributed systems, you will see that the ideal design is often a combination of different pieces. Combining FaaS with more flexible infrastructure or IaaS primitives, like the core of the Kubernetes API, often allows you to gain the best of both possible worlds and blend power and flexibility with simplicity and automation for the optimal system design.

Previous chapter
< 8. Scatter/Gather

Next chapter
10. Ownership Election >